

Vietnam House Price Prediction

Intelligent House Price Prediction System

Objective

Build a machine learning system that predicts the selling price of a house in Vietnam based on its characteristics such as:

- Area
- Frontage
- Access Road
- Floors
- Bedrooms
- Bathrooms
- Location
- House Direction
- Balcony Direction
- Legal Status
- Furniture State

The final selected model will be prepared for deployment on an Android mobile application.

1. Problem Definition

Problem

The objective is to predict the selling price of a house using its physical characteristics and location information.

The target variable is **Price**, measured in billion Vietnamese Dong.

Formal Problem Statement

Given the characteristics of a house, predict its selling price for a previously unseen house.

Learning Task

This is a regression problem because the target **Price** is a continuous numerical value.

2. Dataset Source

Dataset: House Price Prediction Dataset Vietnam - 2024

Source: Kaggle

Kaggle URL: <https://www.kaggle.com/datasets/nguyentiennhan/vietnam-housing-dataset-2024>

The dataset contains real estate listings from Vietnam. Each row represents one house/property listing.

```
In [60]: import pandas as pd
import numpy as np
import matplotlib.pyplot as plt
from sklearn.model_selection import train_test_split
df = pd.read_csv("vietnam_housing_dataset.csv")

# Display the first 5 rows
df.head()
```

Out[60]:

	Address	Area	Frontage	Access Road	House direction	Balcony direction	Floors	Bedrooms	Bathrooms
0	Dự án The Empire - Vinhomes Ocean Park 2, Xã L...	84.0	NaN	NaN	NaN	NaN	4.0	NaN	NaN
1	Dự án The Crown - Vinhomes Ocean Park 3, Xã Ng...	60.0	NaN	NaN	NaN	NaN	5.0	NaN	NaN
2	Dự án The Crown - Vinhomes Ocean Park 3, Xã Ng...	90.0	6.0	13.0	Đông - Bắc	Đông - Bắc	5.0	NaN	NaN
3	Đường Nguyễn Văn Khối, Phường 11, Gò Vấp, Hồ C...	54.0	NaN	3.5	Tây - Nam	Tây - Nam	2.0	2.0	3.0
4	Đường Quang Trung, Phường 8, Gò Vấp, Hồ Chí Minh	92.0	NaN	NaN	Đông - Nam	Đông - Nam	2.0	4.0	4.0

```
In [61]: display(df.head(1).T)
```

0

Address	Dự án The Empire - Vinhomes Ocean Park 2, Xã L...
Area	84.0
Frontage	NaN
Access Road	NaN
House direction	NaN
Balcony direction	NaN
Floors	4.0
Bedrooms	NaN
Bathrooms	NaN
Legal status	Have certificate
Furniture state	NaN
Price	8.6

```
In [62]: # Number of rows and columns
print("Dataset shape:", df.shape)

# Column names
print("\nColumns:")
print(df.columns.tolist())

# Data types and non-null counts
print("\nDataset information:")
df.info()
```

Dataset shape: (30229, 12)

Columns:

['Address', 'Area', 'Frontage', 'Access Road', 'House direction', 'Balcony direction', 'Floors', 'Bedrooms', 'Bathrooms', 'Legal status', 'Furniture state', 'Price']

Dataset information:

<class 'pandas.DataFrame'>

RangeIndex: 30229 entries, 0 to 30228

Data columns (total 12 columns):

#	Column	Non-Null Count	Dtype
0	Address	30229 non-null	str
1	Area	30229 non-null	float64
2	Frontage	18665 non-null	float64
3	Access Road	16932 non-null	float64
4	House direction	8990 non-null	str
5	Balcony direction	5246 non-null	str
6	Floors	26626 non-null	float64
7	Bedrooms	25067 non-null	float64
8	Bathrooms	23155 non-null	float64
9	Legal status	25723 non-null	str
10	Furniture state	16110 non-null	str
11	Price	30229 non-null	float64

dtypes: float64(7), str(5)

memory usage: 5.4 MB

```
In [63]: # Count missing values
missing_count = df.isnull().sum()

# Calculate missing percentage
missing_percent = (
    df.isnull().mean() * 100
).round(2)

missing_table = pd.DataFrame({
    "Missing Count": missing_count,
    "Missing Percentage": missing_percent
})

missing_table.sort_values(
    "Missing Percentage",
    ascending=False
)
```

Out[63]:

	Missing Count	Missing Percentage
Balcony direction	24983	82.65
House direction	21239	70.26
Furniture state	14119	46.71
Access Road	13297	43.99
Frontage	11564	38.25
Bathrooms	7074	23.40
Bedrooms	5162	17.08
Legal status	4506	14.91
Floors	3603	11.92
Area	0	0.00
Address	0	0.00
Price	0	0.00

3. Dataset Description

The dataset contains 30,229 property observations and 12 columns.

Numerical Features

- Area
- Frontage
- Access Road
- Floors
- Bedrooms
- Bathrooms

Categorical Features

- Address
- House direction
- Balcony direction
- Legal status
- Furniture state

Target

- Price

The target is a continuous numerical value measured in billion VND, so the task is a regression problem.

4. Address Feature Engineering

The original `Address` column contains detailed text.

For example:

`Đường Nguyễn Văn Khối, Phường 11, Gò Vấp, Hồ Chí Minh`

Instead of using the entire address as a high-cardinality text feature, we extract two meaningful location features:

- District: Gò Vấp
- City/Province: Hồ Chí Minh

This allows the model to capture regional differences in house prices without creating thousands of unique address categories.

```
In [64]: # Make a copy so the original Address column remains available
df["Address_Copy"] = df["Address"].astype(str)

# Split each address by comma
address_parts = df["Address_Copy"].str.split(",")

# Example:
# "Đường Nguyễn Văn Khối, Phường 11, Gò Vấp, Hồ Chí Minh"
#
# parts:
# [0] = Đường Nguyễn Văn Khối
# [1] = Phường 11
# [2] = Gò Vấp
# [3] = Hồ Chí Minh

# The Last part is the City/Province
df["City_Province"] = (
    address_parts
    .str[-1]
    .str.strip()
)

# The second-last part is the District
df["District"] = (
    address_parts
    .str[-2]
    .str.strip()
)

# Display the result
df[
```

```
    "Address",
    "District",
    "City_Province"
]
].head(20)
```

Out[64]:

	Address	District	City_Province
0	Dự án The Empire - Vinhomes Ocean Park 2, Xã L...	Văn Giang	Hưng Yên
1	Dự án The Crown - Vinhomes Ocean Park 3, Xã Ng...	Văn Giang	Hưng Yên
2	Dự án The Crown - Vinhomes Ocean Park 3, Xã Ng...	Văn Giang	Hưng Yên
3	Đường Nguyễn Văn Khối, Phường 11, Gò Vấp, Hồ C...	Gò Vấp	Hồ Chí Minh
4	Đường Quang Trung, Phường 8, Gò Vấp, Hồ Chí Minh	Gò Vấp	Hồ Chí Minh
5	Dự án The Empire - Vinhomes Ocean Park 2, Xã L...	Văn Giang	Hưng Yên
6	Dự án The Empire - Vinhomes Ocean Park 2, Xã L...	Văn Giang	Hưng Yên
7	Dự án Him Lam Thường Tín, Huyện Thường Tín, Hà...	Huyện Thường Tín	Hà Nội
8	Dự án The Empire - Vinhomes Ocean Park 2, Xã L...	Văn Giang	Hưng Yên
9	Dự án The Crown - Vinhomes Ocean Park 3, Xã Ng...	Văn Giang	Hưng Yên
10	Dự án The Crown - Vinhomes Ocean Park 3, Xã Ng...	Văn Giang	Hưng Yên
11	Dự án The Crown - Vinhomes Ocean Park 3, Xã Ng...	Văn Giang	Hưng Yên
12	Dự án The Crown - Vinhomes Ocean Park 3, Xã Ng...	Văn Giang	Hưng Yên
13	Dự án The Empire - Vinhomes Ocean Park 2, Xã L...	Văn Giang	Hưng Yên
14	Dự án The Empire - Vinhomes Ocean Park 2, Xã L...	Văn Giang	Hưng Yên
15	Đường Phạm Văn Chiêu, Phường 14, Gò Vấp, Hồ Ch...	Gò Vấp	Hồ Chí Minh
16	Dự án Khu đô thị Phương Đông, Xã Đông Xá, Vân ...	Vân Đồn	Quảng Ninh
17	Dự án The Crown - Vinhomes Ocean Park 3, Xã Ng...	Văn Giang	Hưng Yên
18	Dự án The Empire - Vinhomes Ocean Park 2, Xã L...	Văn Giang	Hưng Yên
19	Đường Quang Trung, Phường 12, Gò Vấp, Hồ Chí Minh	Gò Vấp	Hồ Chí Minh

```
In [65]: df.describe(include="all")
```

Out[65]:

	Address	Area	Frontage	Access Road	House direction	Balcony direction	FI
count	30229	30229.000000	18665.000000	16932.000000	8990	5246	26626.00
unique	10265	NaN	NaN	NaN	8	8	
top	Dự án The Empire - Vinhomes Ocean Park 2, Xã L...	NaN	NaN	NaN	Đông - Nam	Đông - Nam	
freq	187	NaN	NaN	NaN	1916	1087	
mean	NaN	68.498741	5.361692	7.853800	NaN	NaN	3.41
std	NaN	48.069835	4.346174	7.451313	NaN	NaN	1.32
min	NaN	3.100000	1.000000	1.000000	NaN	NaN	1.00
25%	NaN	40.000000	4.000000	4.000000	NaN	NaN	2.00
50%	NaN	56.000000	4.500000	6.000000	NaN	NaN	3.00
75%	NaN	80.000000	5.000000	10.000000	NaN	NaN	4.00
max	NaN	595.000000	77.000000	85.000000	NaN	NaN	10.00

```
In [66]: # Display remaining columns
print(df.columns.tolist())
```

['Address', 'Area', 'Frontage', 'Access Road', 'House direction', 'Balcony direction', 'Floors', 'Bedrooms', 'Bathrooms', 'Legal status', 'Furniture state', 'Price', 'Address_Copy', 'City_Province', 'District']

```
In [67]: print("Price statistics:")
print(df["Price"].describe())
```

Price statistics:
count 30229.000000
mean 5.872078
std 2.211877
min 1.000000
25% 4.200000
50% 5.900000
75% 7.500000
max 11.500000
Name: Price, dtype: float64

5. Data Representation

Each house is represented by a combination of numerical and categorical features.

Numerical Representation

- Area
- Frontage
- Access Road
- Floors
- Bedrooms
- Bathrooms

Categorical Representation

- District
- City/Province
- House direction
- Balcony direction
- Legal status
- Furniture state

The categorical features will later be converted into numerical representations using one-hot encoding.

The numerical features will be standardized when appropriate.

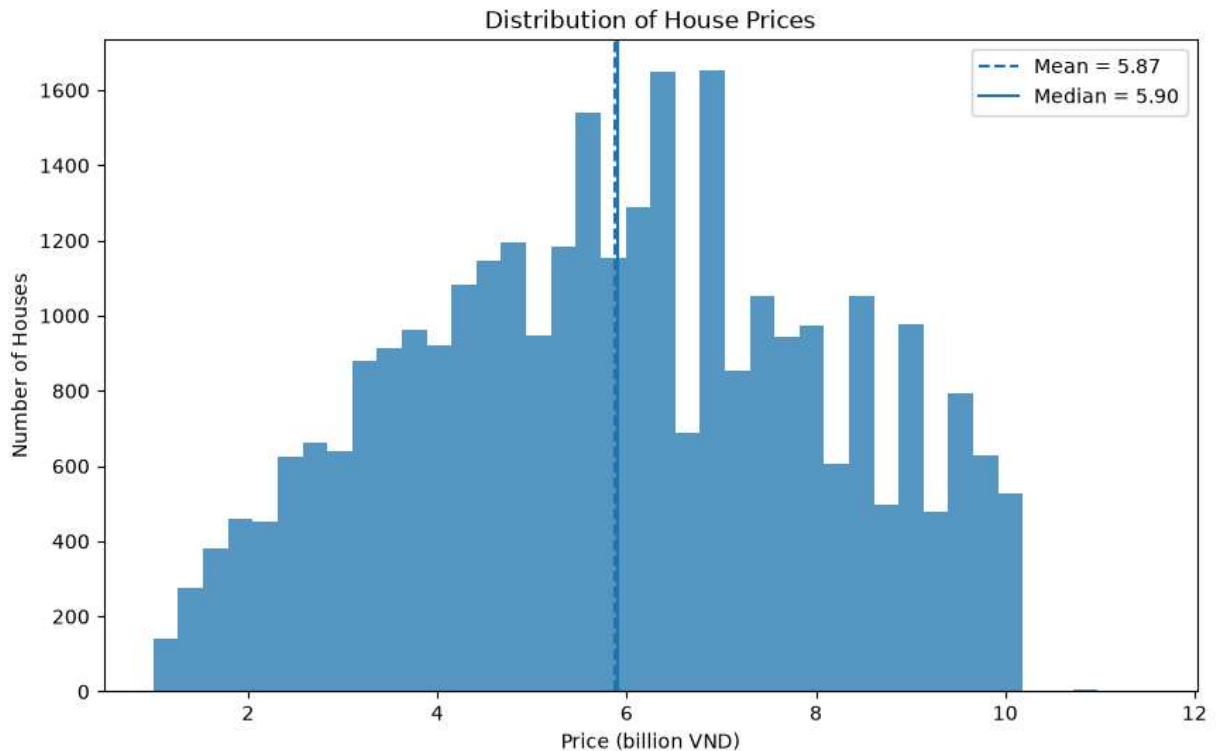
Therefore, the final machine learning input is a numerical feature vector.

```
In [68]: # =====  
# VISUALIZATION 1: PRICE DISTRIBUTION  
# =====  
  
price = df["Price"].dropna()  
  
mean_price = price.mean()  
median_price = price.median()  
  
plt.figure(figsize=(10, 6))  
  
# Histogram of house prices  
plt.hist(  
    price,  
    bins=40,  
    alpha=0.75  
)  
  
# Mean price  
plt.axvline(  
    mean_price,  
    linestyle="--",  
    label=f"Mean = {mean_price:.2f}"  
)
```

```
# Median price
plt.axvline(
    median_price,
    linestyle="--",
    label=f"Median = {median_price:.2f}"
)

plt.title("Distribution of House Prices")
plt.xlabel("Price (billion VND)")
plt.ylabel("Number of Houses")
plt.legend()

plt.show()
```



```
In [69]: # =====
# VISUALIZATION 2:
# AREA VS PRICE
# =====

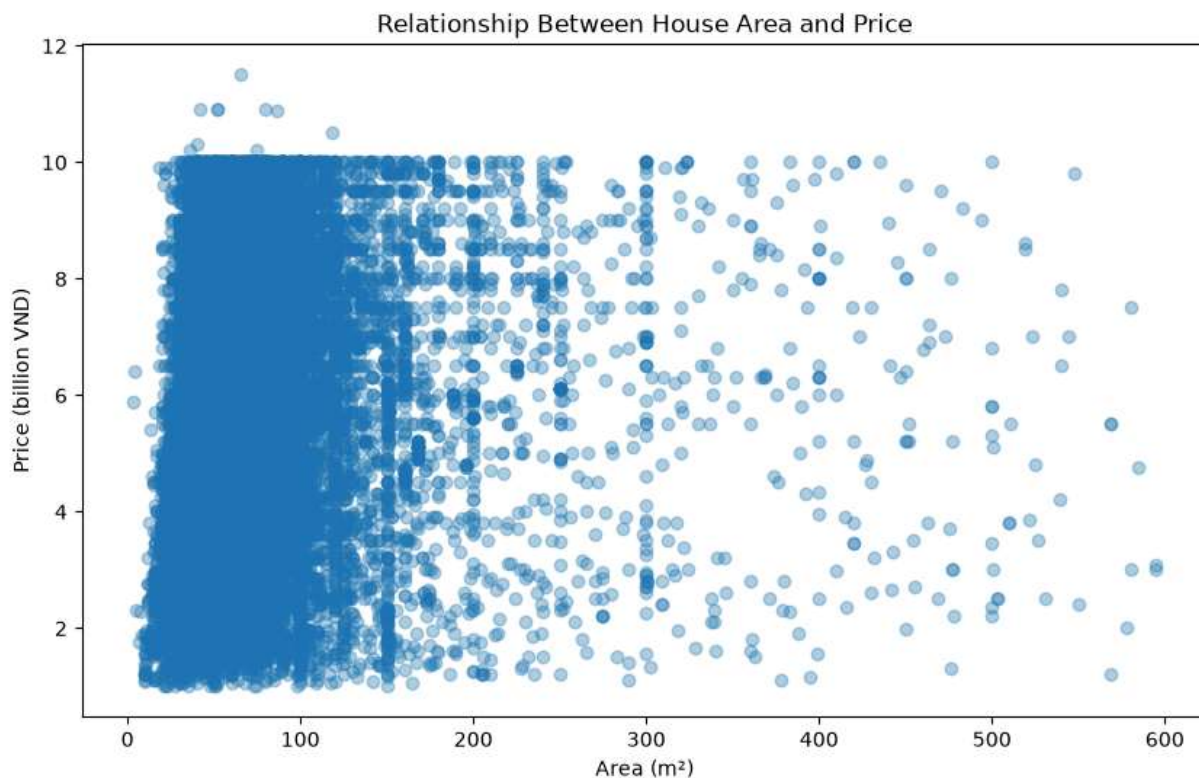
plt.figure(figsize=(10, 6))

plt.scatter(
    df["Area"],
    df["Price"],
    alpha=0.35
)

plt.title(
    "Relationship Between House Area and Price"
)

plt.xlabel("Area (m²)")
plt.ylabel("Price (billion VND)")
```

```
plt.show()
```



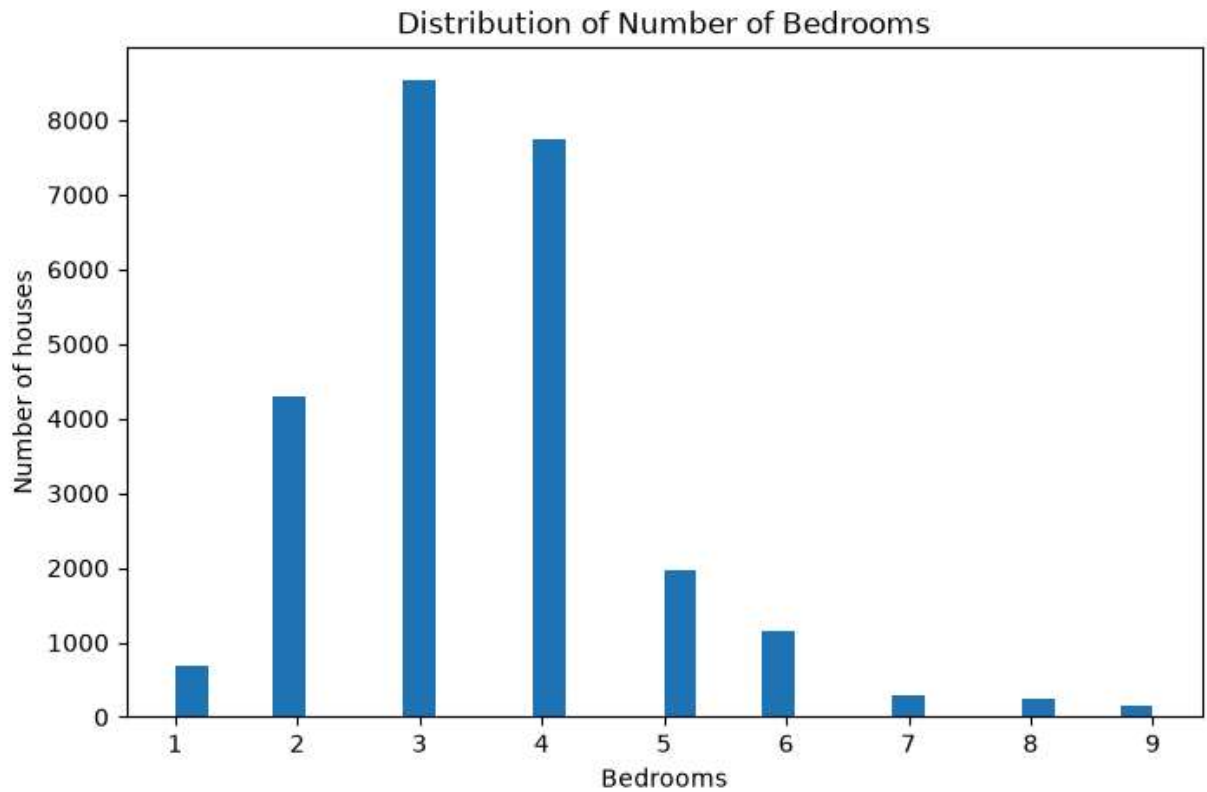
```
In [70]: # =====
# DISTRIBUTION 3: NUMBER OF BEDROOMS
# =====

plt.figure(figsize=(8, 5))

plt.hist(
    df["Bedrooms"].dropna(),
    bins=30
)

plt.title("Distribution of Number of Bedrooms")
plt.xlabel("Bedrooms")
plt.ylabel("Number of houses")

plt.show()
```



6. Visualization Analysis

The three visualizations examine different aspects of the dataset:

1. The price histogram describes the overall distribution of the target.
2. The bedroom boxplot compares house-price distributions across different numbers of bedrooms.
3. The area-price scatter plot examines the relationship between house area and price.

These visualizations are useful before model training because they help us understand the target distribution and important relationships between features and the target.

7. Data Cleaning and Missing Values

The dataset contains a large number of missing values.

Missing values cannot be passed directly to all machine learning models, so they must be handled before training.

For numerical features, the median is used because it is less sensitive to extreme values than the mean.

For categorical features, the most frequent category is used as a simple and reproducible imputation strategy.

```
In [71]: from sklearn.compose import ColumnTransformer
from sklearn.pipeline import Pipeline
from sklearn.impute import SimpleImputer
from sklearn.preprocessing import StandardScaler
from sklearn.preprocessing import OneHotEncoder

numeric_features = [
    "Area",
    "Frontage",
    "Access Road",
    "Floors",
    "Bedrooms",
    "Bathrooms"
]

categorical_features = [
    "District",
    "City_Province",
    "House direction",
    "Balcony direction",
    "Legal status",
    "Furniture state"
]

target = "Price"

X = df[
    numeric_features + categorical_features
]

y = df[target]

print("Number of features:", X.shape[1])
print("Number of samples:", X.shape[0])
```

Number of features: 12
Number of samples: 30229

```
In [72]: from sklearn.model_selection import train_test_split

# Separate the target from the features
X = df.drop(columns=["Price"])
y = df["Price"]

# Split the dataset into training and testing subsets
X_train, X_test, y_train, y_test = train_test_split(
    X,
    y,
    test_size=0.20,      # 20% reserved for testing
    random_state=42      # Reproducible split
)

print("Training samples:", len(X_train))
print("Testing samples:", len(X_test))
```

Training samples: 24183

Testing samples: 6046

8. Preprocessing Strategy

The dataset contains both numerical and categorical features.

Numerical features

Missing values are replaced with the median, then features are standardized.

Categorical features

Missing values are replaced with the most frequent category, then categories are converted into one-hot encoded numerical features.

The preprocessing is implemented inside a Scikit-learn Pipeline so that the same preprocessing learned from the training data can be applied to new data and eventually to the mobile application.

```
In [ ]: # Numerical preprocessing:
# 1. Fill missing values with median
# 2. Standardize numerical features
numeric_pipeline = Pipeline([
    (
        "imputer",
        SimpleImputer(strategy="median")
    ),
    (
        "scaler",
        StandardScaler()
    )
])

# Categorical preprocessing:
# 1. Fill missing values with most frequent category
# 2. Convert categories into numerical one-hot vectors
categorical_pipeline = Pipeline([
    (
        "imputer",
        SimpleImputer(strategy="most_frequent")
    ),
    (
        "encoder",
        OneHotEncoder(
            handle_unknown="ignore"
        )
    )
])

# Combine both preprocessing pipelines
```

```

preprocessor = ColumnTransformer([
    (
        "numeric",
        numeric_pipeline,
        numeric_features
    ),
    (
        "categorical",
        categorical_pipeline,
        categorical_features
    )
])

```

```

In [ ]: # =====
# MODEL 1: LINEAR REGRESSION
# =====

from sklearn.linear_model import LinearRegression

linear_model = Pipeline([
    (
        "preprocessor",
        preprocessor
    ),
    (
        "model",
        LinearRegression()
    )
])

# Train the model
linear_model.fit(
    X_train,
    y_train
)

print("Linear Regression trained.")

```

```

In [ ]: # =====
# MODEL 2: RIDGE REGRESSION
# =====

from sklearn.linear_model import Ridge

ridge_model = Pipeline([
    (
        "preprocessor",
        preprocessor
    ),
    (
        "model",
        Ridge(alpha=1.0)
    )
])

```

```

ridge_model.fit(
    X_train,
    y_train
)

print("Ridge Regression trained.")

```

```

In [50]: # =====
# MODEL 3: SUPPORT VECTOR REGRESSION
# =====

from sklearn.svm import SVR

svr_model = Pipeline([
    (
        "preprocessor",
        preprocessor
    ),
    (
        "model",
        SVR(
            kernel="rbf",
            C=100,
            epsilon=0.1
        )
    )
])

svr_model.fit(
    X_train,
    y_train
)

print("SVR trained.")

```

SVR trained.

```

In [51]: # =====
# MODEL 4: RANDOM FOREST REGRESSOR
# =====

from sklearn.ensemble import RandomForestRegressor

random_forest_model = Pipeline([
    (
        "preprocessor",
        preprocessor
    ),
    (
        "model",
        RandomForestRegressor(
            n_estimators=200,
            random_state=42,
            n_jobs=-1
        )
    )
])

```

```

])

random_forest_model.fit(
    X_train,
    y_train
)

print("Random Forest trained.")

```

Random Forest trained.

```

In [52]: # =====
# MODEL 5: GRADIENT BOOSTING REGRESSOR
# =====

from sklearn.ensemble import GradientBoostingRegressor

gradient_boosting_model = Pipeline([
    (
        "preprocessor",
        preprocessor
    ),
    (
        "model",
        GradientBoostingRegressor(
            n_estimators=200,
            learning_rate=0.05,
            max_depth=3,
            random_state=42
        )
    )
])

gradient_boosting_model.fit(
    X_train,
    y_train
)

print("Gradient Boosting trained.")

```

Gradient Boosting trained.

9. Evaluation Metrics

Five regression metrics are used:

MAE

Mean Absolute Error.

It represents the average absolute prediction error in billion VND.

Lower is better.

MSE

Mean Squared Error.

Large errors are penalized more strongly.

Lower is better.

RMSE

Root Mean Squared Error.

It has the same unit as the target price.

Lower is better.

R²

Coefficient of determination.

It measures how much variation in house prices is explained by the model.

Higher is better.

MAPE

Mean Absolute Percentage Error.

It measures the average prediction error as a percentage.

Lower is better.

```
In [1]: # =====  
# REGRESSION EVALUATION FUNCTION  
# =====  
  
from sklearn.metrics import (  
    mean_absolute_error,  
    mean_squared_error,  
    r2_score  
)  
  
def evaluate_regression_model(  
    model,  
    X_test,  
    y_test  
):  
    """  
    Evaluate a regression model using:  
    MAE, MSE, RMSE, R2 and MAPE.  
    """
```

```

# Predict prices for unseen test data
predictions = model.predict(X_test)

# Mean Absolute Error
mae = mean_absolute_error(
    y_test,
    predictions
)

# Mean Squared Error
mse = mean_squared_error(
    y_test,
    predictions
)

# Root Mean Squared Error
rmse = np.sqrt(mse)

# R-squared
r2 = r2_score(
    y_test,
    predictions
)

# Avoid division by zero when calculating MAPE
non_zero_mask = y_test != 0

mape = (
    np.mean(
        np.abs(
            (
                y_test[non_zero_mask]
                -
                predictions[non_zero_mask]
            )
            /
            y_test[non_zero_mask]
        )
    )
    * 100
)

return {
    "MAE": mae,
    "MSE": mse,
    "RMSE": rmse,
    "R2": r2,
    "MAPE": mape
}

```

```

In [54]: # =====
# EVALUATE ALL FIVE MODELS
# =====

models = {

```

```

    "Linear Regression": linear_model,
    "Ridge Regression": ridge_model,
    "SVR": svr_model,
    "Random Forest": random_forest_model,
    "Gradient Boosting": gradient_boosting_model
}

results = []

for model_name, model in models.items():

    metrics = evaluate_regression_model(
        model,
        X_test,
        y_test
    )

    metrics["Model"] = model_name

    results.append(metrics)

results_df = pd.DataFrame(results)

# Put the model name first
results_df = results_df[
    [
        "Model",
        "MAE",
        "MSE",
        "RMSE",
        "R2",
        "MAPE"
    ]
]

results_df

```

Out[54]:

	Model	MAE	MSE	RMSE	R2	MAPE
0	Linear Regression	1.275297	2.694519	1.641499	0.447382	27.201762
1	Ridge Regression	1.275167	2.683424	1.638116	0.449657	27.166973
2	SVR	1.074609	2.145307	1.464687	0.560020	21.985591
3	Random Forest	1.004896	1.911122	1.382433	0.608048	21.006174
4	Gradient Boosting	1.261667	2.528913	1.590256	0.481346	28.139118

10. Model Comparison

For MAE, MSE, RMSE and MAPE, smaller values indicate better performance.

For R^2 , larger values indicate better performance.

Therefore, the best model should not be selected using only one metric. The results of all five metrics should be considered together.

```
In [55]: # =====  
# FIND BEST MODEL FOR EACH METRIC  
# =====  
  
print("Best MAE:")  
print(  
    results_df.loc[  
        results_df["MAE"].idxmin()  
    ]  
)  
  
print("\nBest MSE:")  
print(  
    results_df.loc[  
        results_df["MSE"].idxmin()  
    ]  
)  
  
print("\nBest RMSE:")  
print(  
    results_df.loc[  
        results_df["RMSE"].idxmin()  
    ]  
)  
  
print("\nBest R2:")  
print(  
    results_df.loc[  
        results_df["R2"].idxmax()  
    ]  
)  
  
print("\nBest MAPE:")  
print(  
    results_df.loc[  
        results_df["MAPE"].idxmin()  
    ]  
)
```

```
Best MAE:
Model      Random Forest
MAE         1.004896
MSE         1.911122
RMSE        1.382433
R2          0.608048
MAPE        21.006174
Name: 3, dtype: object
```

```
Best MSE:
Model      Random Forest
MAE         1.004896
MSE         1.911122
RMSE        1.382433
R2          0.608048
MAPE        21.006174
Name: 3, dtype: object
```

```
Best RMSE:
Model      Random Forest
MAE         1.004896
MSE         1.911122
RMSE        1.382433
R2          0.608048
MAPE        21.006174
Name: 3, dtype: object
```

```
Best R2:
Model      Random Forest
MAE         1.004896
MSE         1.911122
RMSE        1.382433
R2          0.608048
MAPE        21.006174
Name: 3, dtype: object
```

```
Best MAPE:
Model      Random Forest
MAE         1.004896
MSE         1.911122
RMSE        1.382433
R2          0.608048
MAPE        21.006174
Name: 3, dtype: object
```

11. Hyperparameter Experiment

Question

Does changing the number of trees in Random Forest affect prediction performance?

We keep the dataset, preprocessing, training/test split and evaluation procedure unchanged, and only change `n_estimators`.

```

In [56]: # =====
# HYPERPARAMETER EXPERIMENT:
# RANDOM FOREST NUMBER OF TREES
# =====
from sklearn.pipeline import Pipeline
rf_experiment_results = []

for n_trees in [50, 100, 200, 300]:

    model = Pipeline([
        (
            "preprocessor",
            preprocessor
        ),
        (
            "model",
            RandomForestRegressor(
                n_estimators=n_trees,
                random_state=42,
                n_jobs=-1
            )
        )
    ])

    # Train the model
    model.fit(
        X_train,
        y_train
    )

    # Evaluate the model
    metrics = evaluate_regression_model(
        model,
        X_test,
        y_test
    )

    metrics["n_estimators"] = n_trees

    rf_experiment_results.append(
        metrics
    )

rf_experiment_df = pd.DataFrame(
    rf_experiment_results
)

rf_experiment_df

```

Out[56]:

	MAE	MSE	RMSE	R2	MAPE	n_estimators
0	1.013912	1.946262	1.395085	0.600841	21.207512	50
1	1.007486	1.921548	1.386199	0.605910	21.030209	100
2	1.004896	1.911122	1.382433	0.608048	21.006174	200
3	1.005040	1.908459	1.381470	0.608594	21.002784	300

In [57]:

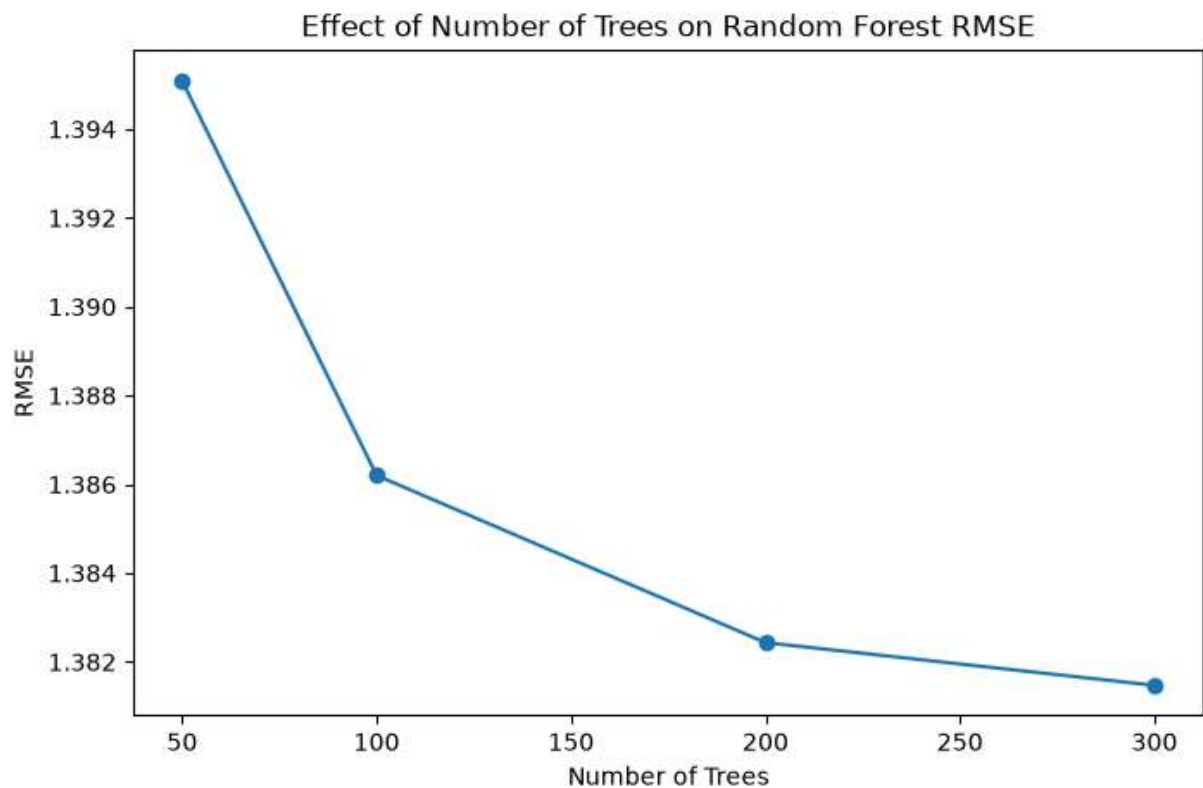
```
# =====
# PLOT RANDOM FOREST HYPERPARAMETER EXPERIMENT
# =====

plt.figure(figsize=(8, 5))

plt.plot(
    rf_experiment_df["n_estimators"],
    rf_experiment_df["RMSE"],
    marker="o"
)

plt.xlabel("Number of Trees")
plt.ylabel("RMSE")
plt.title(
    "Effect of Number of Trees on Random Forest RMSE"
)

plt.show()
```



12. Final Model

After comparing the five regression models and conducting the hyperparameter experiment, the final model is Random Forest selected based on the overall evaluation results.

The final model should provide a good balance between:

- prediction accuracy,
- error magnitude,
- generalization,
- and mobile deployment feasibility.

```
In [58]: # =====  
# SAVE FINAL MODEL  
# =====  
  
import joblib  
  
# Example:  
# Replace `final_model` with the model selected  
# after the actual evaluation.  
  
final_model = random_forest_model  
  
# Save the complete pipeline:  
# preprocessing + trained model  
joblib.dump(  
    final_model,  
    "house_price_model.pkl"  
)  
  
print(  
    "Final model saved to house_price_model.pkl"  
)
```

Final model saved to house_price_model.pkl

13. Mobile Deployment

The trained model is not the final product.

The final system will be deployed as an Android application.

The mobile application will:

1. Receive house information from the user.
2. Convert the input into the same feature representation used during training.
3. Apply the same preprocessing.
4. Run the trained model.

5. Display the predicted house price.

System pipeline:

User Input → Feature Representation → Preprocessing → ML Model → Predicted Price

```
In [59]: rf_model = random_forest_model.named_steps["model"]  
  
         print(rf_model)
```

RandomForestRegressor(n_estimators=200, n_jobs=-1, random_state=42)

```
In [ ]:
```